

UNIVERZA ALMA MATER EUROPAEA

SPLETNE IN INFORMACIJSKE TEHNOLOGIJE

**BREZSTREŽNIŠKA ARHITEKTURA V
OBLAČNEM RAČUNALNIŠTVU**

Mentor: Dušan Fugina

Kandidat: Nuša Dvoršak

Maribor, maj 2026

KAZALO VSEBINE

1 UVOD.....	3
2 KAJ JE BREZSTREŽNIŠKA ARHITEKTURA.....	4
3 KAKO DELUJE BREZSTREŽNIŠKA ARHITEKTURA	5
4 KLJUČNE KOMPONENTE.....	7
4.1 Prednosti	8
4.2 Slabosti	8
5 PRIMER IMPLEMENTACIJE PRI APLIKACIJI ILOGRAPH.....	9
6 ZAKLJUČEK	10
7 SEZNAM LITERATURE IN VIROV	11

KAZALO TABEL

Tabela 1 Primerjava brezstrežniške arhitekture in tradicionalne	6
---	---

KAZALO SLIK

Slika 1 Brezstrežniška infrastruktura	4
Slika 2 Delovanje brezstrežniške infrastrukture.....	5
Slika 3 Delovanje brezstrežniške arhitekture	6

1 UVOD

Olačno računalništvo je v zadnjih letih postalo ena ključnih tehnologij za razvoj sodobnih spletnih aplikacij in informacijskih sistemov. Podjetja in razvijalci vse pogosteje uporabljajo oblačne platforme, saj omogočajo dostop do zmogljive infrastrukture brez potrebe po lastnih strežnikih. Tradicionalni pristopi razvoja aplikacij temeljijo na upravljanju strežnikov, omrežij in drugih infrastrukturnih komponent, kar zahteva precej časa, znanja ter stalnega vzdrževanja. Poleg tega je pri takšnem pristopu potrebno vnaprej načrtovati zmogljivosti sistema, kar lahko vodi bodisi v premalo zmogljivo infrastrukturo bodisi v nepotrebne stroške zaradi prevelikih kapacitet.

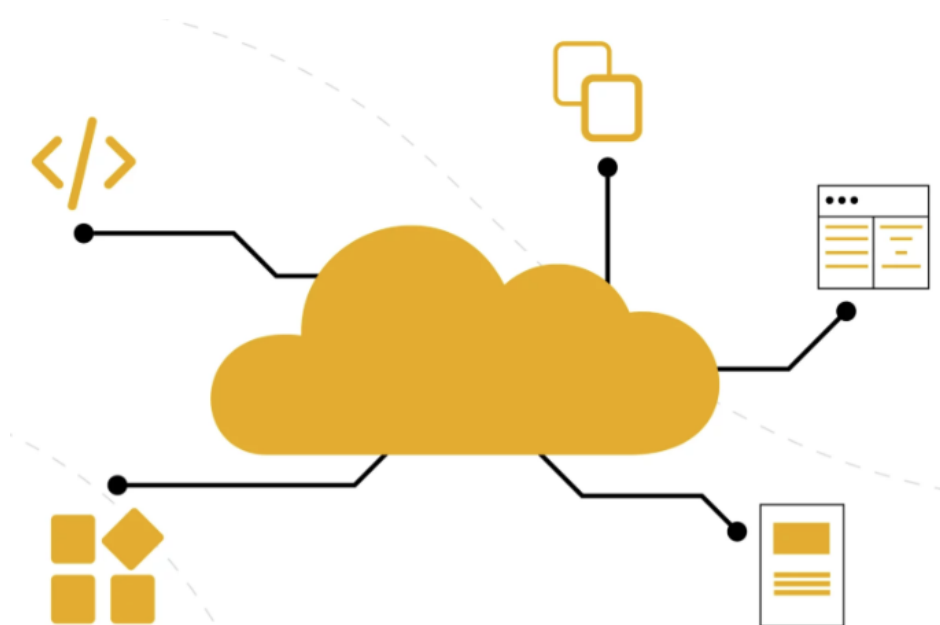
Z razvojem oblačnih tehnologij so se pojavili novi pristopi k razvoju in izvajanju aplikacij. Eden izmed njih je brezstrežniška arhitektura (ang. *serverless architecture*), ki razvijalcem omogoča razvoj aplikacij brez potrebe po neposrednem upravljanju strežniške infrastrukture. Pri tem večino nalog, povezanih z upravljanjem strežnikov, skaliranjem in razpoložljivostjo sistema, prevzame ponudnik oblačnih storitev. Razvijalci se tako lahko bolj osredotočijo na razvoj aplikacijske logike in funkcionalnosti.

V seminarski nalogi je predstavljena brezstrežniška arhitektura, njen način delovanja ter glavne komponente, ki omogočajo izvajanje aplikacij v takšnem okolju. Opisane so tudi glavne prednosti in slabosti tega pristopa ter konkreten primer implementacije pri aplikaciji Ilograph.

2 KAJ JE BREZSTREŽNIŠKA ARHITEKTURA

Brezstrežniška arhitektura (ang. *serverless architecture*) je pristop k razvoju programske opreme v okviru oblačnega računalništva, pri katerem razvijalcem ni treba upravljati osnovne infrastrukture, kot so strežniki, skaliranje ali vzdrževanje sistema. Namesto tega za dodeljevanje virov, upravljanje strežnikov in prilagajanje zmogljivosti skrbi ponudnik oblačnih storitev. Razvijalci se lahko tako osredotočijo predvsem na pisanje in uvajanje kode, ki se izvaja v popolnoma upravljanem okolju (Datadoghq, 2025).

Aplikacijska logika je običajno razdeljena na manjše funkcije, ki se izvajajo v brezstanjskih (ang. *stateless*) okoljih in se sprožijo ob določenih dogodkih, kot so zahteve uporabnikov ali spremembe v sistemu. Takšen model omogoča dinamično prilagajanje infrastrukture in učinkovito uporabo virov, saj oblačni ponudnik samodejno skrbi za skaliranje in razporejanje računalniških zmogljivosti (Datadoghq, 2025).

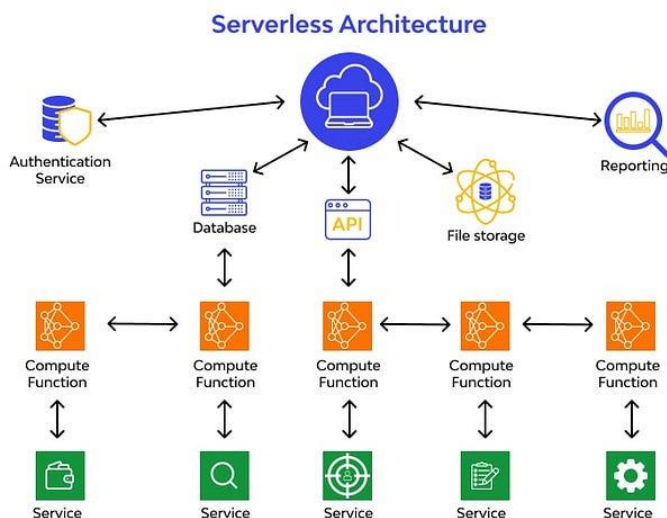


Slika 1 Brezstrežniška infrastruktura

Vir: <https://medium.com/@dave-patten/serverless-architecture-pros-cons-and-use-cases-4a0769744ca2>

3 KAKO DELUJE BREZSTREŽNIŠKA ARHITEKTURA

Strežniki omogočajo uporabnikom dostop do aplikacij in njihove poslovne logike, vendar njihovo upravljanje zahteva veliko časa, znanja in virov. Razvojne ekipe morajo skrbeti za vzdrževanje strojne opreme, nameščanje posodobitev, varnost sistema ter izdelavo varnostnih kopij. Pri brezstrežniški arhitekturi se te naloge prenesejo na ponudnika oblačnih storitev, kar razvijalcem omogoča, da se osredotočijo predvsem na razvoj in izboljševanje aplikacije (Digitalocean, 2023).

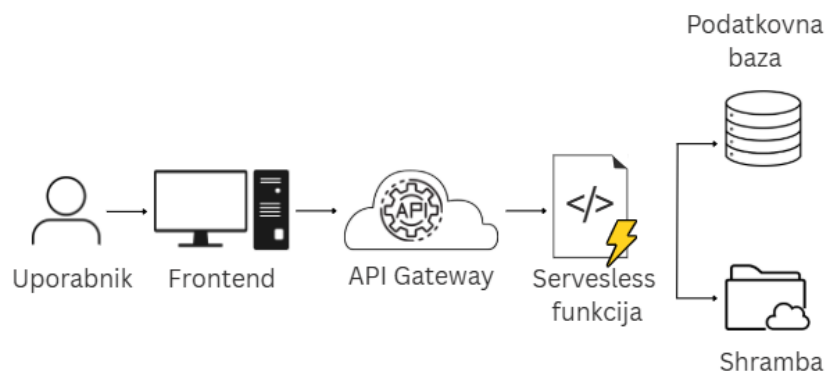


Slika 2 Delovanje brezstrežniške infrastrukture

Vir: <https://medium.com/javarevisited/do-you-know-everything-about-serverless-architecture-f0cd06c81329>

V brezstrežniški arhitekturi uporabnik dostopa do aplikacije preko uporabniškega vmesnika (ang. *frontend*). Ta skrbi za interakcijo z uporabnikom, zajem podatkov in pošiljanje zahtevkov k storitvam v ozadju aplikacije (ang. *backend*). Te zahteve nato preidejo skozi API Gateway, ki deluje kot vmesna plast med frontend-om in backend funkcijami. API Gateway usmerja klice do ustreznih funkcij, skrbi za preverjanje prisotnosti uporabnikov, omejuje število zahtevkov in po potrebi transformira podatke (Digitalocean, 2023).

Dejanska poslovna logika se izvaja v brezstrežniških funkcijah, ki se sprožijo le, ko pride zahteva uporabnika. To omogoča, da aplikacija sama skalira glede na obremenitev, brez da bi morali razvijalci skrbeti za strežniško infrastrukturo. Podatki, ki jih aplikacija uporablja so shranjeni v oblačnih bazah podatkov, medtem ko se večje datoteke, slike ali dokumenti shranjujejo v oblačnih shrambah (Digitalocean, 2023).



Slika 3 Delovanje brezstrežniške arhitekture

Vir: Lasten

Tabela 1 Primerjava brezstrežniške arhitekture in tradicionalne

Lastnost	Brezstrežniška arhitektura	Tradicionalna arhitektura
Upravljanje strežnikov	Upravlja ponudnik oblaka	Upravljanje organizacije
Skaliranje	Samodejno	Ročno
Stroški	Plačilo po porabi	Fiksni stroški
Implementacija	Hitrejša	Počasnejša

Vir: <https://www.digitalocean.com/resources/articles/serverless-architecture>

Iz tabele je razvidno, da brezstrežniška arhitektura poenostavi upravljanje infrastrukture in omogoča bolj fleksibilno uporabo računalniških virov.

4 KLJUČNE KOMPONENTE

Brezstrežniška arhitektura temelji na več ključnih komponentah, ki omogočajo razvoj in delovanje aplikacij brez neposrednega upravljanja strežnikov (Medium, 2024).

Funkcija kot storitev (ang. *Function-as-a-Service* - *FaaS*) je ena izmed ključnih komponent brezstrežniškega računalništva. Gre za model, ki razvijalcem omogoča izvajanje kode v oblaku brez potrebe po upravljanju strežniške infrastrukture. Namesto tega ponudnik oblačnih storitev skrbi za strežnike, razporejanje virov in samodejno skaliranje sistema glede na obremenitev. Funkcije se običajno izvajajo kot odziv na različne dogodke, na primer ob uporabniški zahtevi ali drugi sistemski aktivnosti. Tak pristop razvijalcem omogoča, da se osredotočijo predvsem na razvoj poslovne logike aplikacije, saj platforma samodejno upravlja izvajanje kode in potrebne vire (Medium, 2024).

Brezstrežniške rešitve za shranjevanje podatkov (npr. Amazon S3) omogočajo razširljivo shranjevanje datotek in objektov brez potrebe po upravljanju infrastrukture ter vključujejo šifriranje podatkov v mirovanju (ang. *at rest*), kar preprečuje dostop do informacij v primeru vdora v fizično infrastrukturo. Brezstrežniške podatkovne baze, kot je Amazon DynamoDB, se samodejno prilagajajo obremenitvi in podpirajo tako relacijske (SQL) kot nerelacijske (NoSQL) modele podatkov. Za aplikacije z večjimi programskimi odvisnostmi so na voljo tudi brezstrežniški kontejnerji, ki omogočajo izvajanje kontejnerskih aplikacij brez skrbi glede vzdrževanja strežnikov (Medium, 2024).

Upravljanje API-jev omogoča razvoj, objavo in nadzor nad API vmesniki ter zagotavlja avtentikacijo uporabnikov, omejevanje zahtev in šifriranje med prenosom prek HTTPS. Za povezovanje vseh komponent v enoten proces pa se uporabljajo orodja za orkestracijo, ki poenostavijo razvoj kompleksnih dogodkovno vodenih aplikacij (Medium, 2024).

PREDNOSTI IN SLABOSTI BREZSTREŽNIŠKE ARHITEKTURE

4.1 Prednosti

Brezstrežniške platforme uporabljajo model plačevanja glede na dejansko uporabo (ang. *pay-as-you-go*), kar pomeni, da podjetje plača samo za čas izvajanja funkcij in porabljene vire, ne pa za stalno delovanje strežnikov. Takšen model je posebej ugoden za aplikacije z neenakomernim prometom ali občasno uporabo. Sistem se hkrati samodejno prilagaja obremenitvi, kar omogoča stabilno delovanje tudi ob nenadnih poskokih prometa (Digitalocean, 2023).

Ker ponudnik skrbi za upravljanje strežnikov in posodobitve, se razvijalci osredotočijo na poslovno logiko aplikacije, kar skrajša čas razvoja in pospeši uvajanje novih funkcionalnosti. Brezstrežniška arhitektura je prav tako primerna za dogodkovno vodene sisteme, kjer se funkcije izvajajo kot odziv na dogodke, kot so nalaganje datotek ali podatki iz IoT naprav. Večina storitev je zasnovana z visoko razpoložljivostjo prek porazdeljenih sistemov, kjer ob odpovedi ene komponente delo prevzame druga (Digitalocean, 2023).

4.2 Slabosti

Pri nekaterih funkcijah lahko pride do zamika pri prvem zagonu (ang. *cold start*), ko mora sistem najprej pripraviti izvajalno okolje, kar negativno vpliva na aplikacije z zahtevami po hitrem odzivu (Digitalocean, 2023).

Aplikacije, ki temeljijo na specifičnih storitvah posameznega ponudnika, se težko prenesejo na drugo platformo (vendor lock-in). Brezstrežniške platforme pogosto omejujejo čas izvajanja funkcij in količino pomnilnika, kar je problematično pri dolgotrajnih ali računsko zahtevnih procesih (Digitalocean, 2023).

Ker so aplikacije sestavljene iz več manjših funkcij, je odpravljanje napak bolj zahtevno. Posebno tveganje predstavljajo tudi nepredvidljivi stroški, predvsem t.i. denial-of-wallet napadi, kjer napadalec z množico zahtev ne sesuje sistema, ampak lastniku povzroči ogromne stroške. Razvijalci prav tako nimajo neposrednega nadzora nad konfiguracijo strežnikov ali operacijskega sistema, kar lahko omeji prilagodljivost pri specifičnih zahtevah (Digitalocean, 2023).

5 PRIMER IMPLEMENTACIJE PRI APLIKACIJI ILOGRAPH

Aplikacija Ilograph je interaktivno orodje za ustvarjanje in vizualizacijo diagramov kompleksnih sistemskih arhitektur. Njena zaledna arhitektura temelji na brezstrežniškem pristopu z uporabo storitev AWS (Youtube, 2023).

Uporabniške zahteve se usmerjajo prek storitve Amazon API Gateway, ki sproži ustrezne funkcije v okolju AWS Lambda. Te izvajajo poslovno logiko aplikacije, kot so ustvarjanje, branje, posodabljanje in brisanje podatkov. Diagrami in grafični elementi so shranjeni v storitvi Amazon S3, metapodatki o diagramih pa v podatkovni bazi Amazon DynamoDB. Za upravljanje uporabnikov in avtentikacijo skrbi Amazon Cognito, ki ob prijavi ustvari žeton z informacijami o pravicah dostopa (Youtube, 2023).

Ker so vse komponente nameščene kot ločene oblačne storitve, se sistem samodejno prilagaja številu uporabnikov brez dodatne konfiguracije infrastrukture, kar je dober primer prednosti brezstrežniške arhitekture v praksi (Youtube, 2023).

6 ZAKLJUČEK

Brezstrežniška arhitektura predstavlja sodoben pristop k razvoju aplikacij v oblaknem okolju, ki razvijalcem omogoča, da se bolj osredotočijo na razvoj funkcionalnosti in manj na upravljanje infrastrukture. V nalogi je bilo predstavljeno delovanje brezstrežniške arhitekture ter glavne komponente, ki omogočajo njeno izvajanje. Obravnavane so bile ključne prednosti, kot so samodejno prilagajanje obremenitvam, večja prilagodljivost ter model obračunavanja glede na dejansko uporabo virov, hkrati pa so bili izpostavljeni tudi izzivi – odvisnost od ponudnikov oblčnih storitev ter omejitve pri določenih vrstah aplikacij.

Praktični primer implementacije pri aplikaciji Ilograph je pokazal, kako se brezstrežniška arhitektura uporablja v resničnih sistemih z uporabo storitev AWS Lambda, Amazon S3, DynamoDB in Cognito. Brezstrežniška arhitektura je še posebej primerna za aplikacije s prilagodljivim skaliranjem in neenakomernim prometom ter za mala in srednje velika podjetja, ki želijo razvijati sodobne digitalne storitve brez velikih začetnih investicij v infrastrukturo. Kljub številnim prednostim pa ni vedno najprimernejša rešitev za vse sisteme, zato je pri izbiri arhitekture pomembno upoštevati značilnosti posamezne aplikacije in potrebe organizacije.

7 SEZNAM LITERATURE IN VIROV

1. Be A Better Dev. "Real Life AWS Serverless Architecture Example". YouTube video, 7.38. 16. Okt. 2023. <https://www.youtube.com/watch?v=RBbuERSRU1k>
2. *Datadoghq*. 2026. <https://www.datadoghq.com/knowledge-center/serverless-architecture/#what-is-serverless-architecture>
3. *Digitalocean*. Nazadnje posodobljeno 2023. <https://www.digitalocean.com/resources/articles/serverless-architecture>
4. *Medium*. 2026. <https://medium.com/@dave-patten/serverless-architecture-pros-cons-and-use-cases-4a0769744ca2>